Instructivo de Práctica — Sesión 8: Autenticación segura y CRUD completo

> 📌 **Qué vas a entregar:** al terminar esta práctica, la sección «Gastos del viaje» de ExploraEC funciona de punta a punta contra el backend de gastos: registro e inicio de sesión con JWT, token guardado de forma segura, sesión que sobrevive al reinicio, y crear, editar y eliminar gastos con los errores del servidor a la vista. La **Tarea 3** (se asigna al cierre de esta sesión, se entrega en la Sesión 10) extiende esta misma pantalla con un resumen por categoría. El contrato exacto de la API está en `backend-gastos-referencia.md`.

Objetivo

Reemplazar el cliente `http` de las Sesiones 6-7 por `dio` con interceptores, guardar el token con `flutter_secure_storage`, manejar la sesión con un `AuthController` (GetX) y completar el CRUD de gastos mostrando al usuario los mensajes de error del backend.

Requisitos previos

- Proyecto `exploraec` de la Sesión 7 (lista de gastos con caché de Hive y favoritos persistentes), guardado en git.
- Docker (Docker Desktop en Windows/Mac; Docker Engine con Compose en Linux) y el repositorio del backend `proyecto-curso-spec-kit` de la Sesión 6. Si aún no lo tienes, el Paso 0 trae los comandos.
- Un emulador Android creado (visto en la Sesión 5).

Cómo trabajar esta práctica

Todo el código ya está escrito en la rama de la sesión. **Tú solo descomentas** bloques marcados con `TODO(sesion-08) Paso N`: no hay nada que escribir, copiar ni borrar.

Para descomentar un bloque:

1. Busca el texto `TODO(sesion-08) Paso N` (con `Ctrl + Shift + F` en VS Code).
2. Selecciona las líneas comentadas que están **debajo** del `TODO` (no el `TODO` ni el «Por

qué»).

3. Presiona `Ctrl + /` (`Cmd + /` en Mac). Las líneas quedan sin `//`.

Después de descomentar, en la terminal de `flutter run` presiona `R` (mayúscula, *hot restart*).

Usarás tres terminales (en VS Code: *Terminal → New Terminal*):

Terminal	Carpeta	Para qué
A — backend	`exploraec-curso/proyecto-curso-spec-kit`	`docker compose up` (queda abierta)
B — app	`exploraec-curso/exploraec`	`flutter run` (queda abierta)
C — comandos	`exploraec-curso/exploraec`	`git`, `flutter test`, `curl`, `adb`

Tiempo estimado

Total: 60 minutos

Paso	Descripción	Tiempo
0	Encender todo desde cero y recap de la Sesión 7	10 min
1	Traer la rama de hoy y mirar el cliente HTTP	4 min
2	Iniciar sesión: interceptor del token, `iniciarSesion` y pantalla sin sesión	11 min
3	Crear una cuenta con validación	6 min
4	La sesión sobrevive al reinicio	4 min
5	Crear gastos	7 min
6	Editar y eliminar gastos	5 min
7	Errores: límite de 500, sin conexión y sesión caducada	8 min
8	Cerrar sesión limpia todo	5 min

Paso	Descripción	Tiempo
9 *(opcional)*	Cambiar contraseña — no cuenta dentro de los 60 min	10 min
Cierre	Guardar tu práctica: commit y push — no cuenta dentro de los 60 min	3 min

---

Paso 0 — Encender todo desde cero y recap de la Sesión 7 (10 min)

Por qué se hace

Antes de traer código nuevo hay que tener todo encendido: el backend, el emulador y tu app tal como quedó en la Sesión 7. Así ves, con tus propios ojos, el límite que hoy se resuelve: el token vive solo en memoria y la app te pide iniciar sesión otra vez cada vez que se abre.

Instrucciones

0.1 — Guardar la Sesión 7. En la terminal C:

```
git status
```

Si aparecen cambios sin guardar, guárdalos (el Punto de partida sobrescribe `lib/`):

```
git add -A
git commit -m "Sesión 7"
```

0.2 — Backend. *Solo si no tienes la carpeta* `proyecto-curso-spec-kit` (primera vez en esta computadora), créala desde `exploraec-curso`:

```
git clone https://github.com/cj-murillo/proyecto-curso-spec-kit
cd proyecto-curso-spec-kit
cp .env.example .env
python3 -c "import re,secrets,pathlib; p=pathlib.Path('.env');
p.write_text(re.sub(r'(?m)^SECRET_KEY=.*', 'SECRET_KEY='+secrets.token_hex(32),
p.read_text()))"
```

(En PowerShell, `cp` es `copy .env.example .env`.)

0.3 — Levantar el backend. En la terminal A, dentro de `proyecto-curso-spec-kit`, y déjala abierta:

```
docker compose up
```

La primera vez construye la imagen y tarda varios minutos; las siguientes arranca en segundos. Termina cuando veas `Uvicorn running on http://0.0.0.0:8000`.

0.4 — Comprobar el backend. En la terminal C:

```
curl http://localhost:8000/health
```

Debe responder `{"status":"ok","db":"ok"}`.

0.5 — Encender el emulador. En la terminal C (el `<id>` es la primera columna de la lista):

```
flutter emulators
flutter emulators --launch <id>
```

0.6 — Ejecutar tu app. En la terminal B, con el emulador ya encendido (y déjala abierta):

```
flutter run
```

0.7 — Entrar a tus gastos. En la app: pestaña **Gastos**, inicia sesión con tu correo y tu contraseña, y mira tu lista.

0.8 — Cerrar la app por completo. En la terminal B presiona `q` y desliza la app fuera de recientes del emulador.

0.9 — Abrirla de nuevo. Tócala desde el ícono del emulador y ve a **Gastos**: te pide iniciar

sesión otra vez (el token solo vivía en memoria).

0.10 — Responder en una frase cada una: ¿otro teléfono vería tus gastos guardados en Hive? ¿Qué separa los gastos de dos usuarios, Hive o el servidor? ¿Por qué no guardar el token en Hive?

Resultado esperado

Backend, emulador y app encendidos. Puedes explicar que Hive es una copia local de una persona, que la separación entre usuarios la pone el servidor con el JWT, y que el token necesita un lugar **seguro** para no pedir la contraseña en cada arranque.

Errores comunes

- `Cannot connect to the Docker daemon`. *Solución:* abrir Docker Desktop y esperar a que diga *running*; en Linux, `sudo systemctl start docker`.
- `bind: address already in use` **en el puerto 8000**. *Solución:* otro programa usa el puerto 8000; ciérralo, o detén el backend viejo con `docker compose down` en su carpeta.
- `SECRET_KEY` **vacía** (el backend no arranca). *Solución:* repetir el comando de `python3 -c ...` del punto 0.2 y comprobar con `grep SECRET_KEY .env`.
- `flutter emulators` **no lista ninguno**. *Solución:* crea uno en Android Studio (*Device Manager*), como en la Sesión 5.

✅ Antes de continuar, confirma que...

- ☐ `curl http://localhost:8000/health` responde `ok`.
- ☐ Tu app corre en el emulador y reabrirla te pide iniciar sesión.
- ☐ La Sesión 7 está guardada en un commit.

Punto de partida

Dentro de tu proyecto `exploraec`, en la terminal C. Estos comandos traen el código de hoy (Paso 1 de esta práctica los usa):

```
git fetch starter
git checkout starter/sesion-08 -- lib test pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

La rama trae `dio` y `flutter_secure_storage` en `pubspec.yaml`, el cliente `ApiClient`, el

almacenamiento seguro, las pantallas nuevas y 15 pruebas. Revisa `PLACEHOLDERS.md` para ver la tabla de bloques `TODO` de cada paso.

> ⚠ **Los secretos fuera de git.** Hoy hay dos: el `SECRET_KEY` del backend (solo en su `.env`, que ya está en `.gitignore`) y el **token** (solo en el almacenamiento seguro; nunca en un `print`, un log ni un commit). La dirección del backend se cambia con `--dart-define=API_BASE_URL=...`, sin escribirla fija.

Paso 1 — Traer la rama de hoy y mirar el cliente HTTP (4 min)

Por qué se hace

Hasta ahora cada llamada armaba sus cabeceras a mano con el paquete `http`. Con `dio`, **un solo interceptor** añade el token a todas las peticiones y otro centraliza el caso «token vencido». Antes de activarlos conviene ver dónde viven.

Instrucciones

1. Ejecutar el **Punto de partida** de arriba.
2. Borrar la prueba de la Sesión 7, que ya no aplica (usaba el paquete `http`):

```
rm test/gastos_controller_test.dart
```

(En PowerShell: `del test\gastos_controller_test.dart`.) 3. Comprobar que las pruebas nuevas **fallan a propósito** (aún no descomentas nada):

```
flutter test
```

4. Abrir `lib/services/api_client.dart` y ubicar los dos bloques `TODO(sesion-08)`: el interceptor del token (Paso 2) y el del 401 (Paso 7).
5. En la terminal B, `q` y volver a ejecutar la app (se agregó un plugin nativo nuevo: no basta hot restart):

```
flutter run
```

Resultado esperado

`flutter test` termina con **0 pasando y 15 fallando**. La app arranca igual que en la Sesión 7. Ubicas en `api_client.dart` los dos interceptores comentados.

Errores comunes

- **`flutter test` falla por `package:http`**. *Solución:* quedó la prueba vieja; ejecutar el `rm test/gastos_controller_test.dart` del punto 2.
- **`MissingPluginException` con `flutter_secure_storage`**. *Solución:* detener `flutter run` y ejecutarlo de nuevo desde cero.
- **Error de `minSdkVersion` al compilar**. *Solución:* `flutter_secure_storage` pide Android 6 o superior; revisa `PLACEHOLDERS.md`.

✅ Antes de continuar, confirma que...

- ☐ `flutter test` muestra 15 pruebas fallando.
- ☐ La app arranca sin errores en el emulador.
- ☐ Ubicaste los dos interceptores comentados en `api_client.dart`.

Paso 2 — Iniciar sesión: interceptor del token, `iniciarSesion` y pantalla sin sesión (11 min)

Por qué se hace

El servidor entrega un token al iniciar sesión; desde ahí, cada petición debe llevarlo en la cabecera `Authorization`. El interceptor lo hace solo. El token se guarda en el almacenamiento seguro del sistema (cifrado), no en Hive. Y la sección Gastos solo tiene sentido con sesión: sin ella, muestra un aviso.

Instrucciones

2.1 — Interceptor del token. En `lib/services/api_client.dart`, descomentar el bloque `TODO(sesion-08) Paso 2`.

2.2 — Iniciar sesión. En `lib/controllers/auth_controller.dart`, descomentar el bloque `TODO(sesion-08) Paso 2` (método `iniciarSesion`). Fíjate en que el login se envía como **formulario**, no como JSON.

2.3 — Pantalla sin sesión. En `lib/screens/gastos_screen.dart`, descomentar el bloque

`TODO(sesion-08) Paso 2` (una línea).

2.4 — Aplicar. En la terminal B, presionar `R`.

2.5 — Probar con contraseña incorrecta. En la app: pestaña **Gastos** → **Iniciar sesión** → tu correo y una contraseña equivocada. Aparece el mensaje del servidor bajo el formulario.

2.6 — Probar con la correcta. Escribe tu contraseña real y toca **Entrar**: vuelves a Gastos y ves tu lista.

2.7 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Con la contraseña incorrecta ves *«Email o contraseña incorrectos»*; con la correcta, tu lista. Sin sesión, la pestaña Gastos pide iniciar sesión. `flutter test` da **3 pasando y 12 fallando**.

Errores comunes

- **`422 Unprocessable Entity` al entrar**. *Solución:* se envió JSON; el login exige formulario. Revisa que descomentaste el bloque completo de `iniciarSesion`, sin cambiar nada.
- **La app no cambia tras descomentar**. *Solución:* presiona `R` (mayúscula) en la terminal B; con `r` no basta.
- **`401` con el correo correcto**. *Solución:* revisa la contraseña (mínimo 8 caracteres) y que sea el mismo usuario de la Sesión 6.
- **`Cleartext HTTP traffic not permitted`**. *Solución:* falta `android:usesCleartextTraffic="true"` en `android/app/src/main/AndroidManifest.xml` (lo pusiste en la Sesión 6; solo para desarrollo).
- **El token aparece en la consola**. *Solución:* quitar el `print`: un token en los logs es una fuga de credenciales.

✅ Antes de continuar, confirma que...

- ☐ Una contraseña incorrecta muestra el mensaje del servidor.
- ☐ Con la correcta ves tu lista de gastos.
- ☐ `flutter test` da 3 pasando y 12 fallando.

Paso 3 — Crear una cuenta con validación (6 min)

Por qué se hace

Validar en la app (correo con formato, contraseña de 8 a 72 caracteres, las mismas reglas del backend) evita viajes inútiles al servidor. El servidor sigue siendo la autoridad: si algo falla allá, el mensaje que devuelve se muestra tal cual.

Instrucciones

3.1 — Registrar. En `lib/controllers/auth_controller.dart`, descomentar el bloque `TODO(sesion-08) Paso 3` (método `registrar`).

3.2 — Validadores. En `lib/screens/register_screen.dart`, descomentar los **dos** `TODO(sesion-08) Paso 3` (una línea cada uno: correo y contraseña).

3.3 — Aplicar. En la terminal B, presionar `R`.

3.4 — Cerrar sesión. En la app: menú ⋮ de la barra superior → **Cerrar sesión**.

3.5 — Ir a crear cuenta. Iniciar sesión → **Crear una cuenta**.

3.6 — Contraseña corta. Escribe `abc` de contraseña y toca **Crear cuenta**: el formulario se bloquea sin llamar al servidor.

3.7 — Correo repetido. Con tu correo de siempre y una contraseña válida: aparece el mensaje del servidor (*«El email … ya está registrado»*).

3.8 — Correo nuevo. Con un correo nuevo (por ejemplo `viajero@curso.com`) y una contraseña de 8 o más caracteres: se crea la cuenta y entras.

3.9 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Contraseña corta → bloqueada en la app. Correo repetido → mensaje del servidor. Correo nuevo → cuenta creada y sesión abierta. `flutter test` da **5 pasando y 10 fallando**.

Errores comunes

- **El formulario envía aunque la contraseña sea corta**. *Solución:* falta descomentar alguno de los dos `validator` de `register_screen.dart`.
- **Sale «DioException [bad response]…» en pantalla**. *Solución:* no tocaste el código de

error; si aparece, revisa que descomentaste `registrar` completo.

- **`Undefined name 'Validadores'`**. *Solución:* no borres el `import` de `validadores.dart` que está arriba del archivo.

✅ Antes de continuar, confirma que...

- ☐ Una contraseña de 3 caracteres se rechaza en el formulario.
- ☐ Un correo repetido muestra el mensaje del servidor.
- ☐ Un correo nuevo crea la cuenta y abre la sesión.

Paso 4 — La sesión sobrevive al reinicio (4 min)

Por qué se hace

El token ya se guarda, pero nadie lo lee al abrir la app. Este bloque lo recupera, comprueba con el servidor que siga siendo válido y deja la sesión abierta. Es justo lo que fallaba en el Paso 0.

Instrucciones

4.1 — Restaurar sesión. En `lib/controllers/auth_controller.dart`, descomentar el bloque `TODO(sesion-08) Paso 4` (método `restaurarSesion`).

4.2 — Reiniciar la app por completo. En la terminal B, `q`; desliza la app fuera de recientes; y vuelve a ejecutarla:

```
flutter run
```

4.3 — Comprobar. Ve a la pestaña **Gastos**: entras directo, sin escribir la contraseña.

4.4 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Tras cerrar la app por completo y reabrirla, sigues dentro de la sesión. `flutter test` da **7 pasando y 8 fallando**.

Errores comunes

- **Te pide iniciar sesión de nuevo**. *Solución:* revisa que descomentaste `restaurarSesion` y que cerraste la app por completo (no basta `R`).
- **Se queda un instante con un círculo girando**. *Solución:* es normal: la app está preguntando al servidor si el token sigue siendo válido.
- **Te saca a los 30 minutos «sin motivo»**. *Solución:* el token caduca a los 30 minutos (`ACCESS_TOKEN_EXPIRE_MINUTES=30`); es el comportamiento correcto.

✅ Antes de continuar, confirma que...

- ☐ Al reabrir la app sigues con la sesión iniciada.
- ☐ `flutter test` da 7 pasando y 8 fallando.

Paso 5 — Crear gastos (7 min)

Por qué se hace

Hasta hoy la app solo **leía** gastos. Crear es una llamada distinta (`POST`) y la pantalla debe mostrar lo que el servidor guardó, no suponerlo. Las categorías no se escriben a mano: las define el backend y la app las pide.

Instrucciones

5.1 — Categorías y crear. En `lib/controllers/gastos_controller.dart`, descomentar los **dos** `TODO(sesion-08) Paso 5` (`cargarCategorias` y `crear`).

5.2 — Botón «+». En `lib/screens/gastos_screen.dart`, descomentar el bloque `TODO(sesion-08) Paso 5`.

5.3 — Aplicar. En la terminal B, presionar `R`.

5.4 — Crear un gasto. Toca el botón **+** y completa: descripción `Almuerzo en el Mercado Central`, monto `6.50`, categoría `comida`. Toca **Guardar**: el gasto aparece en la lista.

5.5 — Ver la petición en el servidor. Mira la terminal A: aparece una línea `POST /gastos/ -> 201`.

5.6 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

El gasto nuevo aparece en la lista y el servidor registró un `201`. Las categorías del formulario (`comida`, `transporte`, `entretenimiento`, `otros`) vienen del servidor. `flutter test` da **10 pasando y 5 fallando**.

Errores comunes

- **La lista de categorías sale vacía**. *Solución:* falta el bloque de `cargarCategorias`; descomenta y presiona `R` (las categorías se piden al iniciar sesión).
- `422` **al guardar**. *Solución:* el monto debe ser un número mayor a 0 y la categoría una de las del servidor.
- **No aparece el botón «+»**. *Solución:* falta el bloque del botón en `gastos_screen.dart`.

✅ Antes de continuar, confirma que...

- ☐ Creaste «Almuerzo en el Mercado Central» y aparece en la lista.
- ☐ Viste `POST /gastos/ -> 201` en la terminal A.
- ☐ `flutter test` da 10 pasando y 5 fallando.

Paso 6 — Editar y eliminar gastos (5 min)

Por qué se hace

Editar usa `PATCH` (envía solo los campos que cambiaron) y eliminar usa `DELETE` (responde 204, sin cuerpo). Borrar es irreversible: la app siempre pide confirmación antes.

Instrucciones

6.1 — Controller. En `lib/controllers/gastos_controller.dart`, descomentar los **dos** `TODO(sesion-08) Paso 6` (`actualizar` y `eliminar`).

6.2 — Pantalla. En `lib/screens/gastos_screen.dart`, descomentar los **dos** `TODO(sesion-08) Paso 6` (tocar para editar y botón de eliminar).

6.3 — Aplicar. En la terminal B, presionar `R`.

6.4 — Editar. Toca el gasto, cambia el monto a `7.00` y toca **Guardar cambios**: la lista muestra `7.00`.

6.5 — Cancelar un borrado. Toca el ícono de papelera y luego **Cancelar**: el gasto sigue en la lista.

6.6 — Eliminar. Toca la papelera y luego **Eliminar**: el gasto desaparece.

6.7 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Editar cambia el monto, cancelar no borra, eliminar quita el gasto. `flutter test` da **13 pasando y 2 fallando**.

Errores comunes

- **Tocar el gasto no hace nada**. *Solución:* falta el bloque `onTap` de `gastos_screen.dart`.
- **No aparece la papelera**. *Solución:* falta el bloque del botón de eliminar.
- **Tras eliminar, el gasto reaparece**. *Solución:* la lista se recarga con `GastosRepository` (la caché de la Sesión 7); confirma que descomentaste `eliminar` completo, que ya vuelve a pedir la lista.

✅ Antes de continuar, confirma que...

- ☐ Editaste un monto y se refleja en la lista.
- ☐ Cancelar el borrado no elimina el gasto.
- ☐ `flutter test` da 13 pasando y 2 fallando.

Paso 7 — Errores: límite de 500, sin conexión y sesión caducada (8 min)

Por qué se hace

El backend impone reglas de negocio (máximo **500** por categoría) y responde con un mensaje en español. Una app robusta lo muestra tal cual en vez de «Algo salió mal». También hay que distinguir quién tiene la culpa: 400 es del pedido, 401 es de la sesión, y «sin conexión» no es ninguno de los dos.

Instrucciones

7.1 — Límite de 500. Crea un gasto `Cena grande` de `450`, categoría `comida`. Luego otro `Otra cena` de `100`, también `comida`: el formulario no se cierra y muestra *«Este gasto*

supera el límite de 500.0 para la categoría 'comida'».

7.2 — Sin conexión. En la terminal A presiona `Ctrl + C` para apagar el backend. En la app, con el formulario abierto, toca **Guardar**: aparece *«No hay conexión con el servidor...»*.

7.3 — La caché de la Sesión 7. Vuelve a la lista y toca el ícono de recargar: aparece el aviso *«Sin conexión — gastos guardados»* con tu lista.

7.4 — Volver a encender el backend. En la terminal A, dentro de `proyecto-curso-spec-kit`:

```
docker compose up
```

7.5 — Interceptor del 401. En `lib/services/api_client.dart`, descomentar el bloque `TODO(sesion-08) Paso 7`. En la terminal B, presionar `R`.

7.6 — Provocar un token vencido. En la app: menú ⋮ → **Invalidar token (solo práctica)**, y toca el ícono de recargar. Vuelves a la pantalla sin sesión con el aviso *«Tu sesión caducó, inicia sesión de nuevo.»*.

7.7 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Cada fallo muestra un mensaje distinto y útil: límite superado (texto del servidor), sin conexión, y sesión caducada (con aviso y vuelta a iniciar sesión). Ninguno muestra trazas técnicas como `DioException`. `flutter test` da **14 pasando y 1 fallando**.

Errores comunes

- **El 400 de límite «no ocurre»**. *Solución:* el límite es por categoría y por usuario; suma más de 500 en la **misma** categoría.
- **El mensaje de 400 sale como «Error 400»**. *Solución:* usa el código tal como viene en la rama; no cambies `api_exception.dart`.
- **Tras invalidar el token no pasa nada**. *Solución:* falta el bloque del Paso 7 de `api_client.dart`, o no tocaste recargar después de invalidar.
- **`docker compose up` dice `no configuration file provided`**. *Solución:* ejecútalo dentro de `proyecto-curso-spec-kit`.

✅ Antes de continuar, confirma que...

- ☐ Viste el mensaje del servidor al superar los 500.
- ☐ Sin conexión, la app muestra el aviso y conserva la lista guardada.
- ☐ Un token inválido te lleva a la pantalla sin sesión con el aviso.

Paso 8 — Cerrar sesión limpia todo (5 min)

Por qué se hace

La caché de Hive de la Sesión 7 guarda tus gastos **sin cifrar** en el teléfono. Si dos personas usan el mismo dispositivo y cerrar sesión solo borra el token, la segunda persona podría leer los gastos de la primera. Cerrar sesión debe borrar todo lo que identifica a la persona.

Instrucciones

8.1 — Ver el problema. En la app: inicia sesión de nuevo, y luego menú ⋮ → **Cerrar sesión**.

8.2 — Mirar el disco. En la terminal C (`adb` viene en la carpeta `platform-tools` del SDK de Android):

```
adb shell run-as com.example.exploraec ls app_flutter
```

Siguen apareciendo archivos `gastos_<número>.hive` : los gastos de esa persona se quedaron en el teléfono.

8.3 — Limpiar al cerrar sesión. En `lib/controllers/auth_controller.dart` , descomentar el bloque `TODO(sesion-08) Paso 8` (una línea, dentro de `cerrarSesion`).

8.4 — Aplicar. En la terminal B, presionar `R` .

8.5 — Repetir la prueba. En la app: inicia sesión y luego **Cerrar sesión**. Vuelve a ejecutar el comando de 8.2: el archivo `gastos_<número>.hive` de esa persona ya no aparece.

8.6 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Al cerrar sesión se borran el token, la lista en pantalla y la caja de Hive de esa persona. `flutter test` da **15 pasando y 0 fallando**.

Errores comunes

- **`adb: command not found`**. *Solución:* agrega la carpeta `platform-tools` del SDK de Android al PATH, o ejecuta el comando desde esa carpeta.
- **`run-as: package not debuggable`**. *Solución:* usa la app en modo debug (`flutter run`), no una versión *release*.
- **Siguen apareciendo archivos `gastos_…` de pruebas anteriores**. *Solución:* son de sesiones cerradas antes de este paso; fíjate en que el de la persona que acabas de cerrar ya no está.

✅ Antes de continuar, confirma que...

- ☐ Antes del bloque, el archivo `gastos_<número>.hive` seguía en el teléfono tras cerrar sesión.
- ☐ Después del bloque, el archivo de esa persona desaparece.
- ☐ `flutter test` da 15 pasando.

Paso 9 (opcional) — Cambiar contraseña (10 min)

> Este paso **no cuenta dentro de los 60 minutos** ni bloquea la Sesión 9. Hazlo solo si terminaste antes. Misma mecánica: descomentar y probar.

Por qué se hace

Completa el ciclo de vida de una cuenta y practica un endpoint que responde **204 sin cuerpo** (`PUT /usuarios/me/password`). Con la contraseña actual equivocada el servidor responde con un mensaje que se muestra tal cual.

Instrucciones

9.1 — Controller. En `lib/controllers/auth_controller.dart` , descomentar el bloque `TODO(sesion-08) OPCIONAL Paso 9` (`cambiarPassword`).

9.2 — Menú. En `lib/screens/gastos_screen.dart` , descomentar los **dos** `TODO(sesion-08) OPCIONAL Paso 9` (la opción del menú y la línea que abre la pantalla).

9.3 — Aplicar. En la terminal B, presionar `R`.

9.4 — Contraseña actual equivocada. Menú ⋮ → **Cambiar contraseña**, escribe una contraseña actual incorrecta y una nueva de 8 o más caracteres: aparece el mensaje del servidor.

9.5 — Contraseña actual correcta. Corrige la actual y toca **Cambiar contraseña**: la sesión se cierra. Inicia sesión con la contraseña nueva.

Resultado esperado

Con la contraseña actual equivocada ves el mensaje del servidor; con la correcta, la sesión se cierra y la contraseña antigua deja de funcionar.

Errores comunes

- **No aparece «Cambiar contraseña» en el menú**. *Solución:* falta alguno de los dos bloques de `gastos_screen.dart`.
- `422` **al cambiarla**. *Solución:* la contraseña nueva debe tener de 8 a 72 caracteres.

✅ Antes de continuar, confirma que...

- ☐ Una contraseña actual equivocada muestra el mensaje del servidor.
- ☐ Tras cambiarla, entras con la nueva y la antigua ya no funciona.

Mini-reto *(sin solución escrita)*

Mostrar quién inició sesión. Muestra el correo del usuario como subtítulo de la pantalla Gastos. Pista: el usuario ya está en `AuthController` (`usuario`) y necesita un `Obx`.

Cierre — Guardar tu práctica: commit y push de `exploraec` (3 min)

Por qué se hace

Como en las sesiones anteriores, al final se guarda el trabajo en tu repositorio: queda respaldado y la Sesión 9 parte de un estado conocido (su Paso 0 pide tener la Sesión 8 en un commit). Se guarda **solo la app**; el backend es otro repositorio.

Instrucciones

1. Ver qué cambió (no debe aparecer `proyecto-curso-spec-kit`):

```
git status --short
```

2. Guardar y subir:

```
git add -A
git commit -m "Sesión 8: backend propio con JWT y CRUD de gastos"
git push
```

3. Confirmar:

```
git status -sb
```

Resultado esperado

`git status -sb` muestra `## main...origin/main` **sin** la palabra `ahead`.

Errores comunes

- **`Author identity unknown`**. *Solución:* configurar `git config --global user.name` y `user.email`.
- **`no upstream branch`**. *Solución:* `git push -u origin main`.
- **`rejected`**. *Solución:* `git pull --rebase` y luego `git push`.

✅ Antes de terminar la sesión, confirma que...

- ☐ `git status -sb` muestra `## main...origin/main` sin `ahead`.
- ☐ Detuviste el backend (`Ctrl + C` en la terminal A, luego `docker compose down`) si ya no lo necesitas.

Entregable de la práctica

- `ApiClient` con `dio` e interceptores (token y 401) funcionando.
- Token guardado con `flutter_secure_storage`; la sesión sobrevive al reinicio de la app.
- Registro e inicio de sesión con validación y mensajes específicos.
- Crear, editar y eliminar gastos (con confirmación) contra el backend.
- Límite de 500, sin conexión y sesión caducada mostrados al usuario con mensajes claros.

- Cerrar sesión que borra token, lista y caja de Hive de esa persona.
- Ningún secreto en git (`.env` del backend, token).
- *(Opcional)* Cambiar contraseña (Paso 9).

Conexión con la siguiente sesión

En la Sesión 9 el asistente de IA usa el **resumen de tus gastos**, que ya sabes pedir al backend con tu sesión. El siguiente secreto real es la clave del proveedor de IA: igual que el `SECRET_KEY` de hoy, nunca vive en la app, sino en un servidor intermedio (proxy) que explicaremos desde cero.

Anexo — Todos los comandos de la práctica, en orden

Resumen de consulta (Linux/Mac). Las ediciones de archivo son siempre «descomentar el bloque `TODO(sesion-08) Paso N`»: no son comandos. `<id>` es el que muestra `flutter emulators`.

Paso 0

```
git status
git add -A
git commit -m "Sesión 7"
docker compose up                       # terminal A, en proyecto-curso-spec-kit
curl http://localhost:8000/health       # terminal C
flutter emulators --launch <id>
flutter run                             # terminal B
```

Punto de partida y Paso 1

```
git fetch starter
git checkout starter/sesion-08 -- lib test pubspec.yaml PLACEHOLDERS.md
flutter pub get
rm test/gastos_controller_test.dart
flutter test                            # 0 pasan, 15 fallan
flutter run
```

Pasos 2 a 8 (después de descomentar: `R` en la terminal B)

```
flutter test                            # 3, 5, 7, 10, 13, 14 y 15 pasan al terminar
los Pasos 2 a 8
flutter run                             # solo en el Paso 4 (cierre completo de la app)
docker compose up                       # Paso 7, tras apagar el backend con Ctrl + C
adb shell run-as com.example.exploraec ls app_flutter   # Paso 8, antes y después
```

Cierre

```
git status --short
git add -A
git commit -m "Sesión 8: backend propio con JWT y CRUD de gastos"
git push
git status -sb
```